

- 1) break exits the loop completely, while continue skips the current iteration and continues with the next iteration.
- 2) Switch uses strict equality (===).
- 3) While checks the condition before executing the loop, while do while executes the loop at least once before checking the condition.
- 4) for in iterates over the keys or indexes of an array.
- 5) for of iterates over the values of an array.
- 6) A function declaration is fully hoisted and can be called before its definition. A function expression cannot be used before it is assigned.
- 7) No. An arrow function does not have its own this; it inherits this from its surrounding lexical scope.
- 8) A callback function is a function passed as an argument to another function and executed later.
- 9) A high-order function is a function that takes another function as an argument or returns a function.
- 10) An IIFE (Immediately Invoked Function Expression) is a function that executes immediately after it is defined.
- 11) Base Case
Recursive Case.

- 12) A closure is a function that remembers and can access variables from its outer lexical scope even after the outer function has finished executing.
- 13) Because `var` is function-scoped all callbacks share the same variable, and the loop finishes before the `setTimeout` callbacks execute.
- 14) Global scope can be accessed throughout the program, while block scope can only be accessed inside the block `{ }` where it is declared.
- 15) The scope chain is the process of Javascript searching for a variable from the current scope outward through its parent scope.
- 16) No. Each separate call normally creates its own independent closure and state. They should not share state unless they intentionally reference the same variable or closure instance.
- 17) `Array.reduce()` reduces an array to a single accumulated value.
- 18) It causes infinite recursion until a stack overflow occurs (Maximum call stack size exceeded).
- 19) Variable shadowing occurs when a variable in an inner scope has the same name as a variable in an outer scope.

hiding the outer variable.

20) Because `for...in` iterates over enumerable property keys and may include additional properties, not just array indexes.

Coding:

```
21) function sumDigits(num) {  
    if (num === 0) {  
        return 0;  
    }  
    return (num % 10) + sumDigits(Math.floor(num/10));  
}  
console.log(sumDigits(1234));    O/P: 10.
```

```
22) function CreateCounter(initialValue) {  
    let count = initialValue;  
  
    return function () {  
        count++;  
        return count;  
    };  
}
```

```
const counter1 = CreateCounter(0);  
const counter2 = CreateCounter(10);
```

```
console.log(counter1());
```

```
console.log(counter1());
```

```
console.log(counter2());
```

```
console.log(counter2());
```

O/P

1

2

11

12

Debugging :

```
23) for (var i=0; i < 3; i++) {  
    setTimeout(() => console.log(i), 100);  
}
```

Actual Output:

3
3
3

Because var is function-scoped and all callbacks share the same i variable.

Fixed Code:

```
for (let i=0; i < 3; i++) {  
    setTimeout(() => console.log(i), 100);  
}
```

Correct O/P:

0
1
2

```
24) function sum(n) {
```

```
    if (n === 0) return 0;
```

```
    return n + sum(n);
```

```
}
```

Bug: The recursive call uses `sum(n)`, so `n` never decreases and never reaches the base case.

Fixed Code:

```
function sum(n) {
```

```
    if (n === 0) return 0;
```

```
    return n + sum(n-1);
```

```
} console.log(sum(5));
```

O/P: 15.